

Programação Web I

Introdução ao Javascript

- ✓ O que é JavaScript
- ✓ Formas de usar JavaScript
- ✓ Sintaxe da linguagem
- ✓ Relacionando elementos HTML no código JavaScript
- ✓ Funções nativas
- ✓ Exemplos práticos

Javascript é uma linguagem de programação interpretada criada em 1995, como uma extensão do HTML para o browser Netscape Navigator 2.0. Hoje existem implementações JavaScript nos browsers dos principais fabricantes.

Programas em JavaScript são interpretados linha-por-linha enquanto o browser carrega a página ou executa uma rotina.

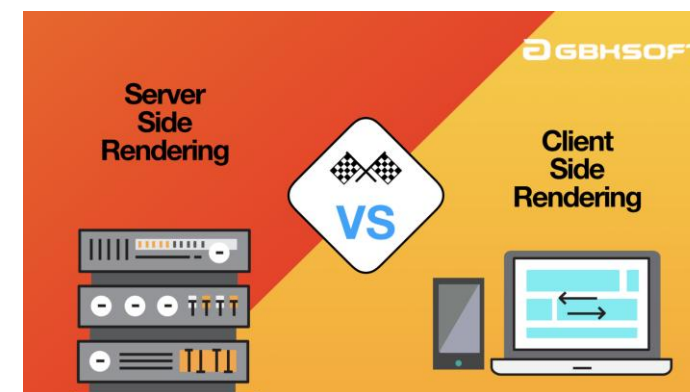
- Linguagem Interpretada
- Simples
- Pequeno
- Extensão do HTML

A JavaScript foi inicialmente implementada e lançada no browser Netscape 2.0, e hoje é a principal linguagem de script do lado cliente, suportada por todos os principais browsers.

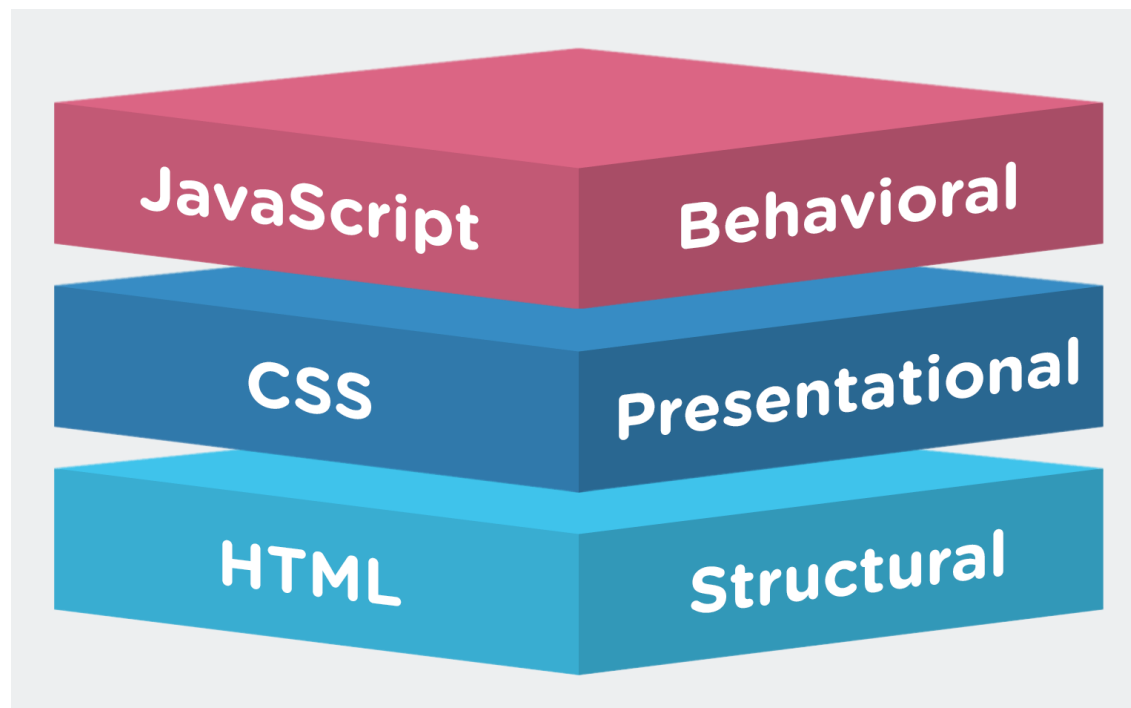
Você pode também ouvir os termos lado do servidor (**server-side**) e lado do cliente (**client-side**), especialmente no contexto de desenvolvimento web.

Códigos do lado do cliente são executados no computador do usuário — quando uma página web é visualizada, o código do lado do cliente é baixado, executado e exibido pelo navegador.

Códigos do lado do servidor, por outro lado, são executados no servidor e o resultado da execução é baixado e exibido no navegador.



JavaScript é baseada em objetos. Trata suas estruturas básicas, propriedades do browser e os elementos de uma página HTML como objetos (entidades com propriedades e comportamentos) e permite que sejam manipulados através de eventos do usuário programáveis, operadores e expressões.



Com JavaScript pode-se fazer diversas coisas que não é possível com HTML:

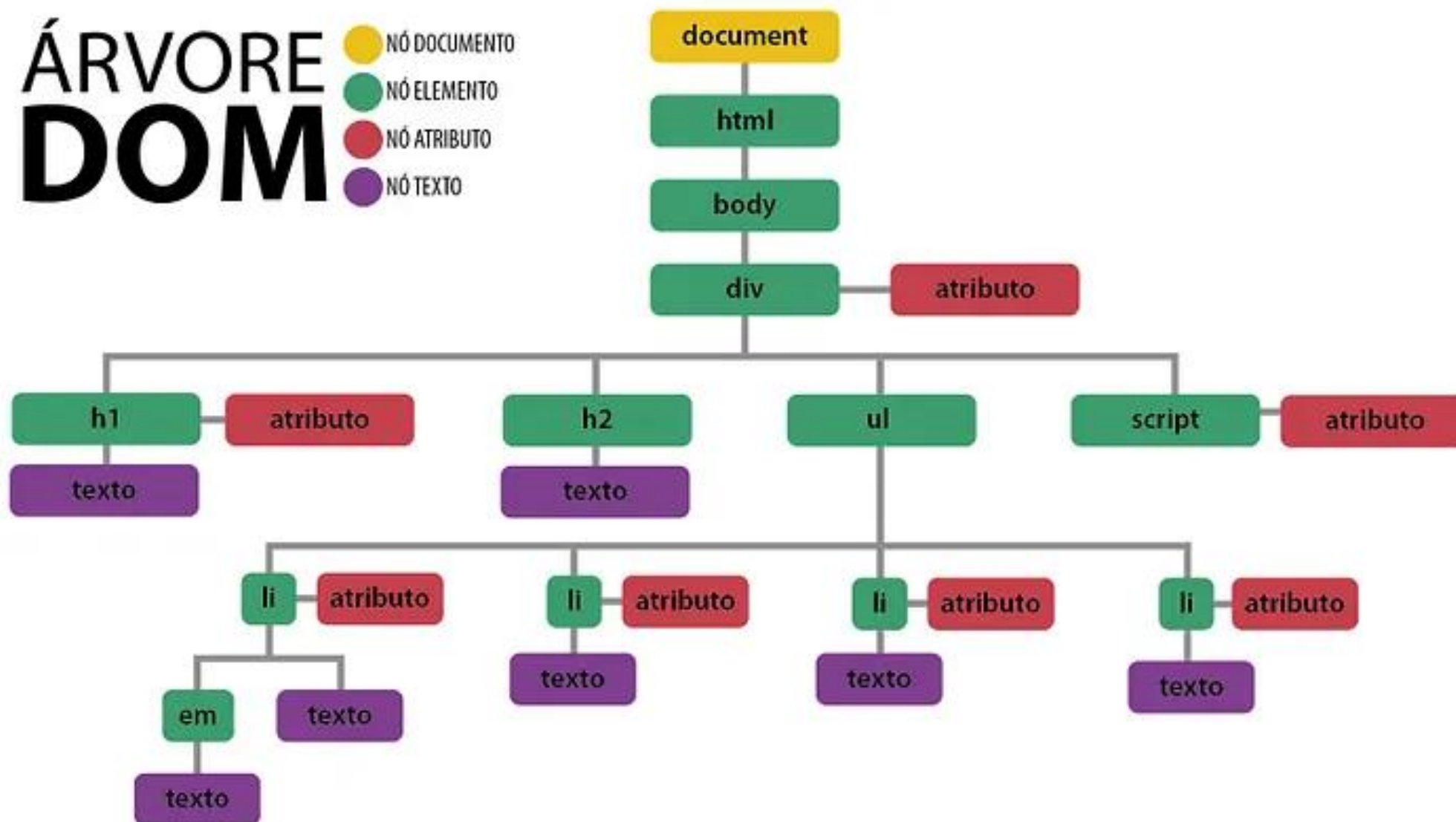
- ☐ Realizar operações matemáticas e computação.
- ☐ Abrir janelas do browser, trocar informações entre janelas, manipular propriedades do browser como o histórico, barra de estado, plug-ins e applets.
- ☐ Interagir com o conteúdo do documento, alterando propriedades da página, dos elementos HTML e tratando toda a página como uma estrutura de objetos.
- ☐ Interagir com o usuário através do tratamento de eventos.

Os códigos JavaScript podem ser inseridos na página HTML de duas formas:

- ☐ Interno no documento: para inserir o código direto na estrutura do HTML, utilizamos as tags `<script>` e `</script>`, conforme abaixo:

ÁRVORE DOM

- NÓ DOCUMENTO
- NÓ ELEMENTO
- NÓ ATRIBUTO
- NÓ TEXTO



- ☐ Externo ao documento: o código JavaScript também pode ser mantido em um arquivo separado do HTML. Para isso, deve-se referenciar tal arquivo na página:

O arquivo deve ser salvo com a extensão .js e sua estrutura é a mesma utilizada quando o código é posto internamente no documento.

Jogo da memória

Iniciar 00:00





css



img



js



index.html



README.md



Roteiro_Jogo_da_Memoria.pdf



teste.html



index.html



styles.css



scripts.js

JavaScript foi criada com base na ECMAScript e sua sintaxe é bastante semelhante a linguagens de alto nível muito utilizadas como C e Java.

Usando variáveis no JavaScript

Essa linguagem possui tipagem dinâmica, ou seja, não é necessário definir o tipo das variáveis ao declará-las, para isso basta usar as palavras reservadas **var (em desuso)**, **let** e **const**.

```
// Declarando variáveis com let
let nome = 'Fulano de Tal';
let soma = 2 + 2;
let status = true;
```

```
// Utilizando const
const nome2 = 'Sicrano';
const calculo = soma + div;
```

O identificador (nome) de uma variável segue a mesma regra de outras linguagens, isto é, deve começar por uma letra ou `_`, os caracteres seguintes podem ser letras, números ou `_` e não pode haver espaços ou caracteres especiais.

- ☐ Uma variável pode ser declarada usando a palavra reservada `let`, `let` ou `const` ou fazendo-se uma atribuição a ela;
- ☐ Sem a palavra reservada (ex: `x = 1;`)
 - ☐ O escopo da variável é global, isto é, ela existirá ao longo de toda a página;
- ☐ Com `let`, `let` ou `const` (ex: `let x;` ou `let x = 1;`)
 - ☐ Caso esteja dentro de uma função, seu escopo é local, isto é, só existirá dentro daquela função.

```
1  <!DOCTYPE html>
2  <html lang="pt-br">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Variáveis</title>
7  </head>
8  <body>
9      <script>
10         var a = 225;
11         let b = 'Fulano de Tal';
12         const Pi = 3.141597;
13         const status = true;
14         // Impressão das variáveis na janela do navegador
15         document.write("O valor de a é "+a+ " e o valor de b é "+b+"<br>");
16         document.write("O valor de Pi é "+Pi+ " e o valor de status é "+status);
17     </script>
18 </body>
19 </html>
```

- A maioria dos operadores de JavaScript são os mesmos de outras linguagens que seguem o padrão ECMA-262 (Java, ActionScript, PHP, C/C++ etc.);
- A seguir, listagem com os principais operadores por grupo e exemplos.

Operador	Significado	Exemplo
+	Soma	5 + 2
++	Incrementa uma unidade	i++
-	Subtração ou inversão de sinal	5 - 2 -3
--	Decrementa uma unidade	i--
*	Multiplicação	5 * 2
/	Divisão	5 / 2
%	Módulo (resto)	5 % 2

Operadores de atribuição

Operador	Significado	Exemplo
=	Recebe (armazena um valor)	x = 3
+=	Recebe o valor dele adicionado com	x += 3
-=	Recebe o valor dele subtraído de	x -= 3
*=	Recebe o valor dele multiplicado por	x *= 3
/=	Recebe o valor dele dividido por	x /= 3
%=	Recebe o resto da divisão dele por	x %= 3

Operadores relacionais

Operador	Significado	Exemplo
>	Maior	5 > 2
<	Menor	5 < 2
>=	Maior ou igual	5 >= 2
<=	Menor ou igual	5 <= 2
==	Igual	5 == 5 5 == "5"
!=	Diferente	5 != 2
===	Igual e do mesmo tipo	5 === "5" 5 === 5

Operador	Significado	Exemplo
&&	E	x && y
	Ou	X y
!	Não	!x

Operadores de concatenação (strings)

Operador	Significado	Exemplo
+	Concatenação	x + y

Operador	Significado	Exemplo
?:	Segundo uma condição, retorna um valor ou outro	a ? b : c
in	Verifica se a propriedade especificada está no objeto especificado	title in plntroducao
instanceof	Verifica se o objeto especificado é daquele tipo	x instanceof String
typeof	Retorna uma string indicando o tipo do operador	typeof "teste"
new	Permite criar um objeto a partir de um construtor	new String("teste")

- ❑ JavaScript aceita os tipos: number (inteiros ou com decimais), string, boolean e null;
- ❑ Entretanto apresenta tipagem fraca, não sendo necessário declarar o tipo de uma variável e a mesma poderá receber valores de um tipo diferente, posteriormente.

Exemplo

```
let x = 0;  
x = "Olá!";  
x = true;  
x = null;
```

Exemplo

```
let x = 0;  
x = "Olá!";  
x = true;  
x = null;
```

Exemplo

```
const x = 0;  
x = "Olá!";  
x = true;  
x = null;
```

Nesse exemplo, a variável x é declarada inicialmente com um valor numérico e, sucessivamente, atribui-se a ela uma string, um boolean e, por fim, null.

❑ Strings

Strings são sequências de caracteres. Em JavaScript a string pode ser tanto um tipo primitivo de dado como um objeto; no entanto, ao manipulá-la temos a impressão de que sejam objetos pois as strings em JavaScript possuem métodos que podemos invocar para realizar determinadas operações sobre elas.

```
// Declarando variáveis com let  
let nome = 'Fulano de Tal';  
let nome2 = "Sicrano";
```

❑ Number

Para uma variável ser tipada como **number**, o conteúdo da mesma não deve estar entre "(aspas simples).

As variáveis numéricas também tem propriedades que permitem que sejam manipuladas conforme a necessidade do código.

```
const myNumber = 23.1608;  
const twoDecimals = myNumber.toFixed(2);  
console.log('Numero com duas casas decimais: ', twoDecimals);
```

CONSOLE ✕

Numero com duas casas decimais: 23.16

❑ Booleano

Uma variável do tipo booleano pode assumir apenas dois valores: true e false. Equivale ao uso de um inteiro com valores 0 ou 1 na linguagem C. O JavaScript converte automaticamente true para 1 e false para 0 quando isso for necessário.

```
var isTrue = true;
if(isTrue === true){
  console.log('Variavel é verdadeira!')
} else {
  console.log('Variavel é falsa!')
}
```

CONSOLE ×

Variavel é verdadeira!

❑ Indefinido

Uma variável é indefinida quando ela foi declarada de alguma forma mas não possui nenhum valor concreto armazenado. Quando tentamos acessar uma variável que não teve nenhum valor associado a ela teremos como retorno "undefined" (indefinido). Exemplo:

```
let name;  
console.log(name);
```

CONSOLE ✕

undefined

❑ Null

O null é a ausência de valor; quando atribuímos null a um objeto ou variável significa que essa variável ou objeto não possui valor válido.

Exemplo:

```
let number = 0;  
console.log(number * 10);
```

CONSOLE ✕

0

Para efeito de comparação, se usarmos o operador de igualdade "=", JavaScript irá considerar iguais os valores null e undefined. No entanto, se for necessário diferenciar os dois valores é recomendável o uso do operador "===" de identidade. Assim, para efeito de comparação, **undefined** e **null** são iguais, mas não idênticos.

Exemplo:

Para **x = 10** temos que :

```
x == 8 -> retorna false  
x == 10 -> retorna true  
x == "10" -> retorna true
```

Para **x = 10** temos que :

```
x === 8 -> retorna false  
x === 10 -> retorna true  
x === "10" -> retorna false
```

Caso seja realizada operação entre string e outro tipo, a conversão dos dados dependerá do operador:

- ❑ Operador suportado por string: converte não string em string;
- ❑ Operador não suportado por string: converte string em não string.

Exemplos

- "casa" + 1 //Resultado será "casa1"
- true + "bola" //Resultado será "truebola"
- "5" * 4 //Resultado será 20

- A maioria dos operadores de JavaScript são os mesmos de outras linguagens que seguem o padrão ECMA-262 (Java, ActionScript, PHP, C/C++ etc.);
- A seguir, listagem com os principais operadores por grupo e exemplos.

Operador	Significado	Exemplo
+	Soma	5 + 2
++	Incrementa uma unidade	i++
-	Subtração ou inversão de sinal	5 - 2 -3
--	Decrementa uma unidade	i--
*	Multiplicação	5 * 2
/	Divisão	5 / 2
%	Módulo (resto)	5 % 2

Exemplo 2 - Operadores

```
1  <!DOCTYPE html>
2  <html lang="pt-br">
3  <head>
4      <meta charset="UTF-8">
5      <meta http-equiv="X-UA-Compatible" content="IE=edge">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Operadores</title>
8
9      <script>
10         // variáveis locais
11         var x = 5;
12         var z = 10;
13         // variáveis globais
14         soma = x + z;
15         subtracao = x - z;
16         mult = x * z;
17         div = x / z;
18         // Impressão dos resultados
19         document.write("O valor da soma é " +soma+ " e a subtração é " +subtracao+ "</br>");
20         document.write("O valor da multiplicação é " +mult+ " e a divisão é " +div+ "</br>");
21     </script>
22 </head>
23 <body>
24 </body>
25 </html>
```

O valor da soma é 15 e a subtração é -5
O valor da multiplicação é 50 e a divisão é 0.5

A linguagem JavaScript possui um número imenso de funções nativas. A primeira função apresentada é a `alert`, que serve para exibir uma mensagem em uma janela **popup**. Essa função recebe apenas um parâmetro, que é o texto a ser exibido.

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="UTF-8"/>
5     <script type="text/javascript">
6       alert("Olá, seja bem vindo. Esta mensagem veio de um alert.");
7     </script>
8   </head>
9   <body>
10  </body>
11 </html>
```

Essa página diz

Olá, seja bem vindo. Esta mensagem veio de um alert.

OK

Em seguida, temos a função `prompt`, que também abre uma janela **popup**, mas com uma caixa de texto para coletar informações do usuário. O retorno dessa função é o texto digitado.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8"/>
    <script type="text/javascript">
      var nome;
      nome = prompt("Qual é seu nome?");
      alert("Olá, " + nome);
    </script>
  </head>
  <body>
  </body>
</html>
```

Essa página diz

Qual é seu nome?

OK

Cancelar

Essa página diz

Olá, Wagner

OK

```
<script type="text/javascript">
```

```
var anoAtual = 2014;
```

```
var anoNascimento = prompt('Digite o ano em que você nasceu.');
```

```
var idade = anoAtual - anoNascimento;
```

```
alert("Sua idade é: " + idade + " anos");
```

```
</script>
```

Essa página diz

Digite o ano em que você nasceu.

OK Cancelar

Essa página diz

Sua idade é: 43 anos

OK

O método **confirm()** é usado para mostrar uma caixa de confirmação, com uma mensagem especificada, e botões OK e Cancelar. Retorna True se o usuário clicar em OK, e False se clicar em Cancelar.

Sintaxe:

confirm("mensagem");

- ❑ mensagem é uma string de texto, opcional, que especifica o texto a mostrar na caixa de confirmação.

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="UTF-8"/>
5      <script type="text/javascript">
6          var decisao = confirm("Clique em um botão!");
7          alert("Você clicou em um botão!");
8      </script>
9  </head>
10 <body>
11 </body>
12 </html>
```

```
1  <!DOCTYPE html>
2  <html lang="pt-br">
3    <head>
4      <metacharset="utf-8">
5      <title>Meu primeiro JavaScript</title>
6      <script type="text/javascript">
7        nome = prompt("Digite seu nome: ", "");
8        document.write("Olá " + nome + " Seja Bem Vindo(a)!");
9      </script>
10   </head>
11   <body>
12   </body>
13 </html>
```

Olá Wagner Seja Bem Vindo(a)!

Essa página diz

Digite seu nome:

Wagner

OK

Cancelar

